

Solucionário – Simulado FGV

11/04/2026

PCUFF

Departamento de Ciência da Computação
Universidade Federal Fluminense

Abril 23, 2026

1. Questão D – *Dislike the Raisins*
2. Questão I – *Integer Multiplicative Persistence*
3. Questão C – *Cut the Deck*
4. Questão E – *E-13 Storage Unit*
5. Questão G – *Grid of Letters*
6. Questão J – *John in the Amusement Park*
7. Questão K – *K-Binary Repetitive Numbers*

2021 ICPC Gran Premio de Mexico 2da Fecha (<https://codeforces.com/gym/103306>)

Link de Submissões

Os links de submissões (nos próximos slides) podem estar quebrados ou bloqueados (por regras do site).

Para ver o código e material, acesse o repositório Git do PCUFF (https://github.com/FelipeHanada/PCUFF/tree/main/Slides/2026_1/23_04_2026)

Questão D – *Dislike the Raisins*

Problema

<https://codeforces.com/gym/103306/problem/D>

Proposição 1

Sempre serão realizadas exatamente $\lceil \frac{C+R}{S} \rceil$ colheradas.

Proposição 2

Para maximizar a quantidade de outstanding spoons, é sempre ótimo pegar o máximo de colheradas cheias de raisins possível. Dessa forma, são realizadas no máximo $\lceil \frac{R}{S} \rceil$ colheradas com raising, portanto, a quantidade máxima de outstanding spoons é $\lceil \frac{C+R}{S} \rceil - \lceil \frac{R}{S} \rceil$.

Questão D – Dislike the Raisins

Proposição 3

Para minimizar a quantidade de outstanding spoons, é sempre ótimo tentar colocar 1 raising por colherada. Podemos fazer isso com $\min \left\{ \lceil \frac{C+R}{S} \rceil, R \right\}$ colheradas, portanto, a quantidade mínima de outstanding spoons é $\lceil \frac{C+R}{S} \rceil - \min \left\{ \lceil \frac{C+R}{S} \rceil, R \right\}$.

Submissão

<https://codeforces.com/gym/103306/submission/370719465>

Questão I – Integer Multiplicative Persistence

Problema

<https://codeforces.com/gym/103306/problem/I>

A ideia chave para resolver essa questão é perceber que:

Proposição 1

A persistência multiplicativa de um número é sempre 'pequena'.

Seja $f(n)$ a persistência multiplicativa de n . O algoritmo que simula o procedimento é capaz de computar $f(n)$ com complexidade $O(f(n) \log n)$, uma vez que cada passo pode ser feito em $O(\log n)$.

Essa constatação seria suficiente para resolver a questão. Mas vamos verificar se $f(n)$ é realmente 'pequeno' o suficiente para fazer nossa solução ser rápida o bastante.

Questão 1 – Integer Multiplicative Persistence

Proposição 2

$$f(n) = O(\log \log n).$$

Demonstração.

Seja n um inteiro com k dígitos, no pior caso, n é formado somente por 9's, e, após a operação, $n' = 9^k$. A quantidade de dígitos de n' é igual à $\log_{10} n' = \log_{10} 9^k = k \log_{10} 9$, assim, a quantidade de dígitos de n é reduzida por uma constante $c = \log_{10} 9$ a cada passo. Como o processo sempre termina quando $k = 1$, podemos garantir que realizaremos no máximo x passos, onde:

$$kc^x = 1 \iff c^x = \frac{1}{k} \iff x = \log_c \frac{1}{k} = -\log_c k = O(\log k)$$

Temos que $k = O(\log n)$. Por fim, $x = O(\log k) = O(\log \log n)$. □

Questão I – *Integer Multiplicative Persistence*

Isso é suficiente para o algoritmo anterior possuir complexidade $O(\log^2 n \log n)$ (**muito rápido!**).

Submissão

<https://codeforces.com/gym/103306/submission/370720325>

Persistência Multiplicativa

Não existem números conhecidos com persistência multiplicativa maior do que 11. Eles não existem até 2.67×10^{30000} .

https://en.wikipedia.org/wiki/Persistence_of_a_number

Questão C – *Cut the Deck*

Problema

<https://codeforces.com/gym/103306/problem/C>

Podemos tomar verificar o resultado de fazer o corte para todo $1 \leq i \leq n$. Resultando em um algoritmo $O(n^2)$, que é lento demais para resolver o problema para as restrições de n . Vamos ver tentar enxergar uma ideia mais refinada.

Condição de vitória de Bob

Seja $prefix(i)$ a soma de todos os elementos do deck até o índice i de forma que $R = -1$ e $B = 1$. Alice ganha se existir $1 \leq i \leq n$ tal que $prefix(i) < 0$. Portanto, desejamos cortar o deck de modo que $prefix$ seja não-negativa após a operação.

Questão C – *Cut the Deck*

Proposição 1

Sempre é possível cortar o deck de modo a fazer Bob ganhar.

Demonstração.

Vamos provar que fazer o corte onde $prefix(i)$ é mínimo é sempre ótimo. Para isso, vamos dar uma olhada no efeito de uma operação sobre os valores de $prefix$. □

Questão C – *Cut the Deck*

Considere o efeito de fazer um corte onde $prefix(y) = x$.

antes: $D_1 \dots D_{y-2} D_{y-1} D_y D_{y+1} D_{y+2} \dots D_n$

depois: $D_{y+1} D_{y+2} \dots D_n D_1 \dots D_{y-2} D_{y-1} D_y$

Como $prefix(y) = x$, no novo deck o $prefix$ de todo índice posterior à y é decrementado em x , ao mesmo tempo, todo índice $\leq y$ deve ser incrementado sobre o novo valor de $prefix(n)$, que é $-x$ (pois era 0 antes da operação), ou seja, também são decrementados em x . Portanto, fazer uma operação em y significa decrementar todo valor de $prefix$ em $prefix(y)$.

Questão C – *Cut the Deck*

Proposição 1

Sempre é possível cortar o deck de modo a fazer Bob ganhar.

Demonstração.

Vamos provar que fazer o corte onde $prefix(i)$ é mínimo é sempre ótimo. Como fazer o corte em $prefix(i)$ decrementa $prefix(i)$ em todo $prefix$, fazer o corte onde $prefix(i)$ é mínimo deixa todos os valores de $prefix$ não-negativos. Portanto, Bob ganha. □

Questão C – *Cut the Deck*

Computar os valores de *prefix* para todo o deck inicial pode ser feito em $O(n)$ e pode ser feito enquanto se procura pelo índice de valor mínimo de *prefix*. Portanto, o algoritmo final roda em $O(n)$.

Submissão

<https://codeforces.com/gym/103306/submission/370719910>

Questão E – E-13 Storage Unit

Problema

<https://codeforces.com/gym/103306/problem/E>

Este problema possui duas soluções bem intuitivas:

Ideia 1: Usar *sliding window*.

Ideia 2: Usar *binary search*.

Questão E – E-13 Storage Unit – Sliding Window

Seja $A = a_1 \dots a_N$ e C o array com o tamanho de todos os n arquivos (em MB) e o tamanho total do disco (em MB), respectivamente.

Para cada i , podemos computar o tamanho da maior 'janela' que não excede o tamanho do disco, isto é, o maior j tal que

$$\sum_{k=i}^j a_k \leq C$$

para todo $1 \leq i \leq N$, em $O(N)$ utilizando *sliding window*.

Questão E – E-13 Storage Unit – Sliding Window

A partir daí, podemos calcular o maior r que satisfaz

$$\sum_{k=i}^{i+r-1} a_k \leq C$$

para todo $1 \leq i \leq N - r + 1$. Iterando da esquerda para a direita mantendo o tamanho da menor janela encontrada R_{min} até que $i + R_{min} - 1 > N$. R_{min} ao final deste processo é o maior valor de r que satisfaz a condição.

Por que?

Questão E – E-13 Storage Unit – Sliding Window

Proposição 1

$R_{min} = R$. Onde R é o maior r que satisfaz a condição.

Demonstração.

Suponha por absurdo que $R_{min} \neq R$, logo, $R \leq R_{min}$ ou $R > R_{min}$.

Se $R < R_{min}$, então R não é máximo, pois R_{min} satisfaz a condição e $R_{min} > R$.

Se $R > R_{min}$, como R satisfaz a condição, então existe uma posição i tal que $i + R - 1 > N$ e $R_j \geq R$ ($j \leq i$). Portanto, o procedimento para calcular R_{min} pararia em i , e, portanto, $R_{min} \geq R$ (o que contradiz a suposição).

(R_i é a maior janela que começa na posição i)

Portanto, $R_{min} = R$. □

Questão E – E-13 Storage Unit – Sliding Window

Para computar L , podemos fazer uma pequena modificação no nosso procedimento de *sliding window*. De modo que guardamos o menor i tal que $R_i = R$ e $i + R \leq N$. Assim, $L = i$ se tal i existir, e $L = -1$ caso contrário.

Como só executamos um algoritmo de *sliding window*, a complexidade final da solução é de $O(N)$.

Submissão

<https://codeforces.com/gym/103306/submission/370737510>

Questão E – E-13 Storage Unit – Binary Search

A segunda ideia se baseia em você notar que:

Proposição 1

Seja $f(r)$ uma função que retorna *true* se 'independente do primeiro vídeo escolhido, o disco irá suportar pelo menos r vídeos' e *false* caso contrário. f é **monotônica**.

Monotonicidade

Uma função f é dita *monotônica crescente* se para todo par $x \leq y$ tem-se que $f(x) \leq f(y)$. E *monotônica decrescente* se para todo par $x \leq y$ tem-se que $f(x) \geq f(y)$.

É um caso de uso **muito** comum do algoritmo de busca binária: encontrar o maior/menor índice $\geq y$ ou $\leq y$ sobre uma função monotônica.

<https://usaco.guide/silver/binary-search>

https://en.wikipedia.org/wiki/Monotonic_function

Questão E – *E-13 Storage Unit* – Binary Search

A monotonicidade de f pode ser facilmente verificada. O problema de encontrar o maior r tal que $f(r) = \text{true}$ para f monotônica é uma aplicação clássica de *busca binária* (como dito anteriormente).

Portanto, se podemos avaliar $f(r)$ em $O(t(r))$, então temos a complexidade $O(t(r) \log N)$. Vamos fornecer um algoritmo que computa $f(r)$ em $O(N)$.

Questão E – E-13 Storage Unit – Binary Search

Prefix Sums

Tome a seguinte função:

$$p(i) = \begin{cases} 0 & i = 0 \\ \sum_{j=1}^i a_j & \text{c.c} \end{cases}$$

A soma do intervalo $\sum_{i=l}^r a_i = p(r) - p(l - 1)$.

Podemos calcular $p(i)$ para todo $1 \leq i \leq N$ em $O(N)$. Para um dado r , podemos iterar sobre todos os i 's e verificar em tempo constante se $\sum_{j=i}^{i+r-1} a_j$ excede C . Portanto, podemos verificar a validade de um r qualquer em $O(N)$. Além disso, podemos computar L facilmente, procurando o primeiro i tal que $\sum_{j=i}^{i+r} a_j$ excede C .

Resultando em uma complexidade final $O(N \log N)$.

Questão G – *Grid of Letters*

Problema

<https://codeforces.com/gym/103306/problem/G>

Ideia: Vamos interpretar a grid como um grafo e buscar o maior caminho.

Construímos um grafo direcionado $G = (V, E)$:

- $v_{i,j} \in V$ para todo $1 \leq i \leq n, 1 \leq j \leq m$
- $v_{i,j}v_{i',j'} \in E$ se:
 - (i,j) e (i',j') são adjacentes na grid (8-direções);
 - e a letra da célula (i',j') sucede a letra da célula (i,j) .

Questão G – *Grid of Letters*

A	B	H
F	F	G
B	D	E
A	F	C

Figura: Grid de entrada.

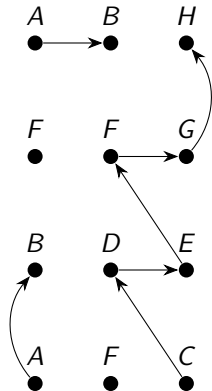


Figura: Grafo G.

Questão G – *Grid of Letters*

Proposição 1

O maior caminho em G é a maior sequência alfabética da grid. Parte diretamente da construção de G .

Proposição 2

G é uma DAG (Directed Acyclic Graph).

- *G é direcionado, pela construção.*
- *G é acíclico. A segunda condição para as arestas confere uma ordenação dos vértices que impede ciclos de ocorrerem.*

Questão G – *Grid of Letters*

O algoritmo de maior caminho para grafos é bem conhecido e pode ser implementado em DAG's com complexidade $O(|V| + |E|)$ aplicando DP (*Dynamic Programming*).

Como $|V| = O(nm)$ e $|E| = O(nm)$ (pois o subjacente de G é uma floresta), a solução final pode ser implementada em $O(nm)$.

Questão G – *Grid of Letters* – Solução

Relação de DP

Seja $dp(v)$ = tamanho do maior caminho que termina no vértice v .

$$dp(v) = \begin{cases} 1 & d^-(v) = 0 \\ \max_{uv \in E} \{dp(u) + 1\} & \text{c.c.} \end{cases}$$

Portanto, o maior caminho de G é o maior valor de $dp(v)$ para todo $v \in V$.

Note que o valor de $dp(v)$ depende do valor do resultado $dp(u)$ de outros vértices, logo, como devemos ordenar os vértices para calcular o valor de $dp(v)$ sem causar problemas de dependência?

Questão G – *Grid of Letters* – Solução 1

Nesta primeira solução, vamos usar a ordenação natural que o problema nos dá: as letras das células. Na próxima solução, generalizaremos esta ordenação para depender apenas da topologia do grafo, ao invés de uma informação adicional do problema.

Proposição 3

Se a letra da célula (i, j) antecede a letra da célula (i', j') , então $dp(v_{i,j})$ não depende de $dp(v_{i',j'})$.

Segue diretamente da segunda condição para a existência da aresta.

Submissão

<https://codeforces.com/gym/103306/submission/370720959>

Questão G – *Grid of Letters* – Solução 2 (Algoritmo de Kahn)

Aqui, aplicamos um algoritmo conhecido para obter uma ordenação topológica de um grafo – o *Algoritmo de Kahn*. A partir desta ordenação, seguimos com a solução discutida.

Submissão

<https://codeforces.com/gym/103306/submission/370668514>

https://en.wikipedia.org/wiki/Topological_sorting#Kahn's_algorithm

Questão J – *John in the Amusement Park*

Problema

<https://codeforces.com/gym/103306/problem/J>

Ideia: Utilizar programação dinâmica.

Seja $dp(t)$ = maior *happiness* capaz de ser obtida até o tempo t de modo que John esteja livre. O problema se torna computar $dp(\infty)$.

Podemos iterar sobre todos os tempos onde John pode fazer uma escolha e computar quando esta escolha melhora um resultado futuro?

Sim! Como a quantidade de atividades é $O(A)$ e cada atividade repete $O(t_i)$ vezes, John pode realizar $O(A \max t_i)$ decisões. Tornando nosso algoritmo viável para as restrições.

Questão J – *John in the Amusement Park*

Inicializamos $dp(0) = 0$ e $dp(t) = -\infty$ para todo $t \neq 0$.

Depois iteramos de forma ordenada sobre todos os momentos t_i onde podemos começar uma atividade. Atualizando $dp(t)$ com $dp(t')$ caso $t' < t$ e $dp(t') > dp(t)$. Isso pode ser feito somente sobre o maior $t' < t$.

E sobre todas as atividades $a_j = (h_j, d_j, t_i)$ que gera h_j *happiness* e demora d_j segundos para ser concluída. Atualizando, para cada atividade, o valor de:

$$dp(t') := \max \{ dp(t'), dp(t_i) + h_j \}$$

para o menor $t' \geq t_i + d_j$ que é um tempo de início de atividade ou ∞ .

Encontrar tal t' pode ser feito em $O(\log(A \max t_i))$, resultando em uma solução com complexidade $O(n \log n)$.

Submissão

<https://codeforces.com/gym/103306/submission/370722490>

Questão K – *K-Binary Repetitive Numbers*

Problema

<https://codeforces.com/gym/103306/problem/K>

Todo número k -binário repetitivo, pode ser construído a partir da repetição de uma sequência binária de tamanho $d < k$ com $d|k$.

Obter a quantidade de sequências de tamanho d para todo divisor de k é muito fácil (pode ser feito em $O(\sqrt{k})$). A dificuldade da questão está em não contar o mesmo número k -binário repetitivo mais de uma vez.

Note que, para $k = 8$, 01010101 é uma sequência válida tanto para $d = 2$ (01010101) quanto $d = 4$ (01010101).

Questão K – *K-Binary Repetitive Numbers*

Ideia: Vamos contar somente as sequências de tamanho d que não possuem uma 'sequência interna', ou seja, que não são d -binário repetitivas.

Proposição 4

Seja $g(k)$ = número de sequências binárias de tamanho k que não são k -binário repetitivas. Então,

$$g(1) = 0$$
$$g(k) = 2^k - \sum_{d < k, d|k} g(d)$$

Resposta

Note que a resposta do problema passa a ser, simplesmente:

$$ans(k) = 2^k - g(k)$$

Questão K – *K-Binary Repetitive Numbers*

Demonstração.

Vamos provar a expressão anterior. Para tal, é suficiente mostrar que as sequências k -binária repetitivas são biunívocas com as sequências de tamanho $d|k$ que não são d -binário repetitivas.

Considerando uma sequência A k -binário repetitiva. Vamos provar que sempre existe uma sequência de tamanho $d|k$ que constrói A por repetição tal que não é d -binário repetitiva, e que essa sequência é única. □

Questão K – *K-Binary Repetitive Numbers*

Demonstração.

Continuação.

1. (Existência) Como A é k -binário repetitiva, existe uma sequência B de tamanho $d|k$ que compõe A por repetição. Caso a sequência B seja d -binário repetitiva, podemos tomar d como o tamanho de qualquer bloco repetitivo de B . Esse procedimento pode ser repetido até que B não seja mais d -binário repetitivo.
2. (Unicidade) Suponha que existem sequências de tamanho $d_1, d_2 < k$ que não são d_i -binário repetitivas que compõe A por repetição, e que, sem perda de generalidade, $d_1 < d_2$. Pela natureza da periodicidade, existe uma sequência de tamanho $d_3 = \gcd(d_1, d_2)$ que compõe A por repetição. Como $d_3 \leq d_1 < d_2$ e $d_3|d_2$, a sequência de tamanho d_2 é d_2 -binário repetitiva, pois pode ser composta pela sequência de tamanho d_3 . Portanto, não existem duas sequências não binário repetitivas que formam A .



Questão K – *K-Binary Repetitive Numbers*

Por fim, podemos precomputar $g(k)$ para todo $1 \leq k \leq 10^6$ em $O(k \log \log k)$ ao modificar uma implementação do crivo de Erathostenes. E responder cada teste em $O(1)$.

Submissão

<https://codeforces.com/gym/103306/submission/370842941>